

# PE-EC702C

## Neural Network & Fuzzy Logic Control

---

### Combined Module Notes

Modules 1 – 5

Semester VII • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

#### Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

**Biraj Sarkar**

IN ASSOCIATION WITH

MAKAUT Wingman • MAKAUT Future Minds

**Cooch Behar Government Engineering College**

*Exam-Ready Notes*

## Contents

---

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Module 1: Biological vs. Artificial Neural Networks</b> | <b>4</b>  |
| <b>1 Biological vs. Artificial Neural Networks</b>         | <b>4</b>  |
| 1.1 Structural and Functional Mapping                      | 4         |
| <b>2 Typical Architectures of ANNs</b>                     | <b>5</b>  |
| <b>3 Common Activation Functions</b>                       | <b>5</b>  |
| <b>4 McCulloch-Pitts Neuron Model</b>                      | <b>6</b>  |
| 4.1 Mathematical Formulation                               | 6         |
| 4.2 Linear Separability and the XOR Problem                | 7         |
| <b>5 Hebb Net Learning Rule</b>                            | <b>7</b>  |
| 5.1 Hebb Net Training Algorithm                            | 7         |
| <b>6 Perceptron Learning Algorithm</b>                     | <b>8</b>  |
| <b>7 Adaline and Madaline Networks</b>                     | <b>8</b>  |
| 7.1 Adaline (Adaptive Linear Neuron)                       | 8         |
| 7.2 Madaline (Many Adalines)                               | 8         |
| <b>8 Pattern Association Networks</b>                      | <b>9</b>  |
| 8.1 Classification of Associative Memories                 | 9         |
| 8.2 Training Rules for Pattern Association                 | 9         |
| <b>9 Bidirectional Associative Memory (BAM)</b>            | <b>9</b>  |
| <b>10 Worked Examples</b>                                  | <b>10</b> |
| <b>Module 2: Introduction to Competitive Learning</b>      | <b>14</b> |
| <b>1 Introduction to Competitive Learning</b>              | <b>14</b> |
| <b>2 Kohonen Self-Organizing Maps (SOM)</b>                | <b>14</b> |
| 2.1 Kohonen SOM Learning Algorithm                         | 14        |
| <b>3 Learning Vector Quantization (LVQ)</b>                | <b>15</b> |
| 3.1 LVQ Update Rules                                       | 15        |
| <b>4 Counter Propagation Networks (CPN)</b>                | <b>16</b> |
| 4.1 CPN Architecture                                       | 16        |
| 4.2 CPN Training phases                                    | 16        |
| <b>5 Worked Examples</b>                                   | <b>16</b> |
| <b>Module 3: Backpropagation Neural Networks (BPNN)</b>    | <b>21</b> |
| <b>1 Backpropagation Neural Networks (BPNN)</b>            | <b>21</b> |
| 1.1 Derivation of the Backpropagation Learning Rules       | 21        |
| <b>2 Adaptive Resonance Theory (ART)</b>                   | <b>22</b> |
| 2.1 ART1 vs. ART2                                          | 23        |

|                                                             |                                                             |           |
|-------------------------------------------------------------|-------------------------------------------------------------|-----------|
| 2.2                                                         | ART1 Architecture and Operation . . . . .                   | 23        |
| <b>3</b>                                                    | <b>Boltzmann Machine Learning . . . . .</b>                 | <b>23</b> |
| 3.1                                                         | Boltzmann Learning Rule . . . . .                           | 24        |
| <b>4</b>                                                    | <b>Worked Examples . . . . .</b>                            | <b>24</b> |
| <b>Module 4: Classical (Crisp) Sets vs. Fuzzy Sets</b>      |                                                             | <b>28</b> |
| <b>1</b>                                                    | <b>Classical (Crisp) Sets vs. Fuzzy Sets . . . . .</b>      | <b>28</b> |
| 1.1                                                         | Standard Fuzzy Operations . . . . .                         | 28        |
| <b>2</b>                                                    | <b>Fuzzy Relations and Composition . . . . .</b>            | <b>28</b> |
| 2.1                                                         | Tolerance and Equivalence Relations . . . . .               | 29        |
| <b>3</b>                                                    | <b>Membership Functions and Standard Forms . . . . .</b>    | <b>29</b> |
| 3.1                                                         | Key Geometric Features . . . . .                            | 29        |
| 3.2                                                         | Standard Shapes . . . . .                                   | 30        |
| <b>4</b>                                                    | <b>Fuzzification and Defuzzification . . . . .</b>          | <b>30</b> |
| 4.1                                                         | Standard Defuzzification Methods . . . . .                  | 30        |
| <b>5</b>                                                    | <b>Lambda Cuts (<math>\lambda</math>-cuts) . . . . .</b>    | <b>31</b> |
| <b>6</b>                                                    | <b>Worked Examples . . . . .</b>                            | <b>31</b> |
| <b>Module 5: Applications of Artificial Neural Networks</b> |                                                             | <b>36</b> |
| <b>1</b>                                                    | <b>Applications of Artificial Neural Networks . . . . .</b> | <b>36</b> |
| <b>2</b>                                                    | <b>Applications of Fuzzy Logic . . . . .</b>                | <b>36</b> |
| <b>3</b>                                                    | <b>Fuzzy Logic Controllers (FLC) . . . . .</b>              | <b>36</b> |
| 3.1                                                         | Standard FLC Architecture Components . . . . .              | 37        |
| <b>4</b>                                                    | <b>Control System Case Studies . . . . .</b>                | <b>37</b> |
| 4.1                                                         | Inverted Pendulum Controller . . . . .                      | 37        |
| 4.2                                                         | Fuzzy Washing Machine Controller . . . . .                  | 37        |
| <b>5</b>                                                    | <b>Worked Examples . . . . .</b>                            | <b>38</b> |

# Module 1

## Biological vs. Artificial Neural Networks

### 1. Biological vs. Artificial Neural Networks

An Artificial Neural Network (ANN) is an information-processing paradigm inspired by the structure and functional aspects of biological neural systems.

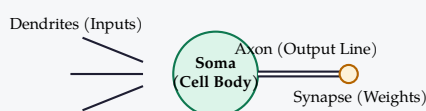
#### Artificial Neural Network (ANN)

An **Artificial Neural Network (ANN)** is a parallel, distributed computational model consisting of simple processing units (neurons) interconnected by weighted links, designed to learn patterns and relationships from data.

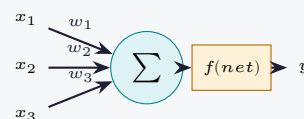
#### 1.1 Structural and Functional Mapping

The biological brain consists of billions of highly interconnected cells called neurons. The artificial neuron models this biological mechanism:

#### Biological vs. Artificial Neuron Mapping



**Biological Neuron Elements**



**Artificial Neuron Model**

The physical correspondences between biological and artificial components are summarized below:

| Biological Component | Artificial Equivalent       | Functional Role                              |
|----------------------|-----------------------------|----------------------------------------------|
| Dendrites            | Inputs ( $x_i$ )            | Receives signals from other neurons          |
| Soma (Cell Body)     | Summing Junction ( $\sum$ ) | Aggregates all incoming weighted inputs      |
| Axon                 | Output Transmission Line    | Carries the output activation signal         |
| Synapse              | Weights ( $w_i$ )           | Regulates the strength of connections        |
| Synaptic Threshold   | Activation Function ( $f$ ) | Determines output state based on total input |

## 2. Typical Architectures of ANNs

Neural network topologies are broadly classified into three categories based on the flow of signals and feedback loops:

1. **Single-Layer Feedforward Network:** Consists of an input layer of source nodes that connects directly to an output layer of neurons. There are no feedback loops and no hidden layers. (Example: Hebb net, Perceptron, Adaline).
2. **Multi-Layer Feedforward Network:** Contains one or more hidden layers of processing units between the input and output layers. This architecture enables the network to learn non-linear decision boundaries. (Example: Backpropagation MLP).
3. **Recurrent (Feedback) Network:** Contains at least one feedback loop where output signals are fed back as inputs to neurons in previous or current layers. (Example: Hopfield Network, BAM).

## 3. Common Activation Functions

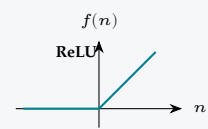
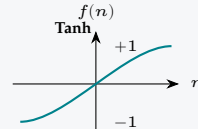
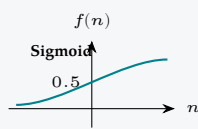
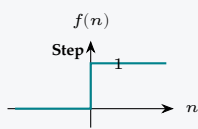
Activation functions (transfer functions) introduce non-linearity into the neuron model, allowing it to solve complex, non-linear problems.

### Mathematical Formulations of Activation Functions

Let  $net = \sum w_i x_i + b$  be the net input to the neuron.

| Function                | Mathematical Form                                                                         | Range         |
|-------------------------|-------------------------------------------------------------------------------------------|---------------|
| Binary Step             | $f(net) = \begin{cases} 1 & \text{if } net \geq 0 \\ 0 & \text{if } net < 0 \end{cases}$  | $\{0, 1\}$    |
| Bipolar Step            | $f(net) = \begin{cases} 1 & \text{if } net \geq 0 \\ -1 & \text{if } net < 0 \end{cases}$ | $\{-1, 1\}$   |
| Binary Sigmoid          | $f(net) = \frac{1}{1+e^{-\lambda net}}$                                                   | $(0, 1)$      |
| Bipolar Sigmoid         | $f(net) = \frac{2}{1+e^{-\lambda net}} - 1 = \tanh\left(\frac{\lambda net}{2}\right)$     | $(-1, 1)$     |
| Rectified Linear (ReLU) | $f(net) = \max(0, net)$                                                                   | $[0, \infty)$ |

### Activation Function Shapes



## 4. McCulloch-Pitts Neuron Model

The McCulloch-Pitts (M-P) neuron, introduced in 1943, was the first mathematical model of an artificial neuron.

### McCulloch-Pitts Neuron

An **M-P Neuron** is a binary threshold device where inputs are either 0 or 1, weights are fixed integers (usually +1 for excitatory and -1 for inhibitory), and the output is determined by comparing the weighted sum against a threshold  $\theta$ .

#### 4.1 Mathematical Formulation

The activation of an M-P neuron is governed by:

$$y = f(net) = \begin{cases} 1 & \text{if } net \geq \theta \\ 0 & \text{if } net < \theta \end{cases} \quad (1)$$

where the net input  $net$  is computed as:

$$net = \sum_{i=1}^n w_i x_i \quad (2)$$

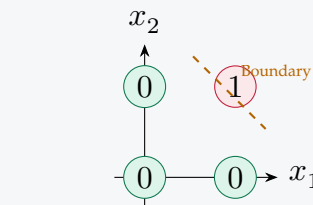
**Inhibitory Rule:** If any inhibitory input  $x_j$  is active (i.e.  $x_j = 1$  where  $w_j = -1$  or  $-\infty$ ), the output  $y$  is forced to 0 immediately, regardless of the excitatory sum.

## 4.2 Linear Separability and the XOR Problem

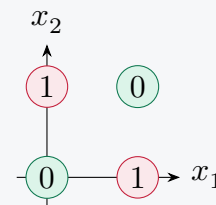
A classification problem is **linearly separable** if a single straight line (in 2D), plane (in 3D), or hyperplane (in higher dimensions) can completely separate the output classes.

Simple logic gates like AND, OR, and NOT are linearly separable. However, the XOR (Exclusive OR) gate is **not** linearly separable, as a single line cannot separate the outputs  $\{0\}$  and  $\{1\}$ . This limitation led to the development of multi-layer neural networks.

### Linear Separability of AND vs. XOR



AND Gate (Linearly Separable)



XOR Gate (Non-Linearly Separable)

## 5. Hebb Net Learning Rule

Proposed by Donald Hebb in 1949, this is the oldest learning algorithm. It is based on the biological premise: *"Neurons that fire together, wire together."*

### Hebbian Learning Rule

If two connected neurons are active simultaneously, the weight of the connection between them should be increased:

$$w_i(\text{new}) = w_i(\text{old}) + x_i y \quad (3)$$

where  $x_i$  is the input and  $y$  is the output.

### 5.1 Hebb Net Training Algorithm

1. Initialize all weights  $w_i = 0$  and bias  $b = 0$ .
2. For each training input vector  $s = (s_1, s_2, \dots, s_n)$  and target output  $t$ :
  - Set activation of input units  $x_i = s_i$ .
  - Set activation of output unit  $y = t$ .
  - Update weights:  $w_i(\text{new}) = w_i(\text{old}) + x_i y$ .
  - Update bias:  $b(\text{new}) = b(\text{old}) + y$ .

*Note: Hebb net learning works best when inputs and targets are represented in bipolar form  $\{-1, 1\}$  rather than binary form  $\{0, 1\}$ .*

## 6. Perceptron Learning Algorithm

Developed by Frank Rosenblatt in 1958, the Perceptron is a supervised learning model used for binary classification.

#### Perceptron Activation & Weight Update

The output is computed using a binary/bipolar step activation:

$$y = \begin{cases} 1 & \text{if } \sum w_i x_i + b \geq \theta \\ -1/0 & \text{if } \sum w_i x_i + b < \theta \end{cases} \quad (4)$$

If the computed output  $y$  does not match the target  $t$ , the weights and bias are updated using the Perceptron Learning Rule:

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y)x_i \quad (5)$$

$$b(\text{new}) = b(\text{old}) + \alpha(t - y) \quad (6)$$

where  $\alpha \in (0, 1]$  is the learning rate.

## 7. Adaline and Madaline Networks

### 7.1 Adaline (Adaptive Linear Neuron)

Developed by Widrow and Hoff (1960), Adaline is a single-element network that utilizes the Delta Rule (LMS - Least Mean Squares) for training.

#### Adaline vs. Perceptron

Unlike the Perceptron where weights are updated based on the thresholded binary output, Adaline updates weights using the continuous analog output **before** the activation function is applied.

The weight update in Adaline minimizes the mean squared error (gradient descent):

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - \text{net})x_i \quad (7)$$

where  $\text{net} = \sum w_i x_i + b$  is the analog pre-activation sum.

### 7.2 Madaline (Many Adalines)

Madaline consists of a multilayer organization of Adaline units. The input layer connects to a hidden layer of Adaline units, which in turn connect to Madaline output units using fixed logic functions (AND, OR, or Majority vote). Madaline is trained using MRI (Madaline Rule I) or MRII algorithms.

## 8. Pattern Association Networks

Pattern association involves storing set of pattern pairs  $(s^p, t^p)$  such that when an input pattern  $s^p$  is presented, the network recalls the associated target pattern  $t^p$ .

### 8.1 Classification of Associative Memories

1. **Auto-Associative Net:** A network where the input vector and target vector are identical ( $s^p = t^p$ ). Its primary goal is pattern retrieval, denoising, or pattern completion. If a distorted or incomplete version of pattern  $s^p$  is presented, the network outputs the original stored pattern  $s^p$ .
2. **Hetero-Associative Net:** A network where the input vector and target vector are different ( $s^p \neq t^p$ , and they may reside in spaces of different dimensions). It maps an input vector in space  $X$  to an associated target vector in space  $Y$ .
3. **Iterative Auto-Associative Net:** A recurrent auto-associative network where output states are fed back as inputs dynamically. The network iteratively updates its activation values until it settles (converges) into a stable state representing one of the stored patterns (e.g., Hopfield networks).

### 8.2 Training Rules for Pattern Association

Training an associative memory network involves constructing a weight matrix  $W$  to map inputs to targets:

1. **Hebbian Learning Rule (Outer Product Rule):** For  $P$  bipolar pattern pairs  $(s^p, t^p)$  where  $s^p \in \{-1, 1\}^n$  and  $t^p \in \{-1, 1\}^m$ , the weight matrix is formed by summing the outer products of input-target pairs:

$$W = \sum_{p=1}^P (s^p)^T t^p \quad \Rightarrow \quad w_{ij} = \sum_{p=1}^P s_i^p t_j^p \quad (8)$$

2. **Delta Rule (LMS / Widrow-Hoff Rule):** Used when patterns are not orthogonally separable. It is an iterative gradient descent rule that updates weights based on output error to minimize mean squared error:

$$\Delta w_{ij} = \alpha(t_j^p - y_j^p)s_i^p \quad (9)$$

where  $\alpha$  is the learning rate,  $t_j^p$  is the target, and  $y_j^p$  is the actual output.

## 9. Bidirectional Associative Memory (BAM)

Introduced by Bart Kosko in 1988, BAM is a hetero-associative recurrent network that maps input vectors from space  $X$  to target vectors in space  $Y$ .

#### BAM Weight Matrix and Recall

Given  $P$  bipolar pattern pairs  $(S^p, T^p)$  where  $S^p \in \{-1, 1\}^n$  and  $T^p \in \{-1, 1\}^m$ :

The weight correlation matrix  $W$  of size  $n \times m$  is computed as:

$$W = \sum_{p=1}^P (S^p)^T T^p \quad (10)$$

**Bidirectional Recall Process:**

- **Forward Direction:** Feed input vector  $X$  to recall output  $Y$ :

$$y_j = \text{sign} \left( \sum_{i=1}^n x_i w_{ij} \right) \rightarrow Y = \text{sign}(XW) \quad (11)$$

- **Backward Direction:** Feed output vector  $Y$  back to recall input  $X$ :

$$x_i = \text{sign} \left( \sum_{j=1}^m y_j w_{ij} \right) \rightarrow X = \text{sign}(YW^T) \quad (12)$$

## 10. Worked Examples

### McCulloch-Pitts AND gate Implementation

**Problem:** Implement an AND gate using a McCulloch-Pitts neuron with binary inputs  $x_1, x_2 \in \{0, 1\}$ . Determine the weights and threshold  $\theta$ .

**Solution:** Let the weights be  $w_1 = 1, w_2 = 1$  (excitatory weights). The net input is  $net = w_1 x_1 + w_2 x_2 = x_1 + x_2$ . The output equation is  $y = 1$  if  $net \geq \theta$ , else 0. Let's construct the truth table and formulate inequalities for  $\theta$ :

| $x_1$ | $x_2$ | Target $t$ | $net = x_1 + x_2$ | Condition for threshold $\theta$ |
|-------|-------|------------|-------------------|----------------------------------|
| 0     | 0     | 0          | 0                 | $0 < \theta$                     |
| 1     | 0     | 0          | 1                 | $1 < \theta$                     |
| 0     | 1     | 0          | 1                 | $1 < \theta$                     |
| 1     | 1     | 1          | 2                 | $2 \geq \theta$                  |

From the table, the threshold  $\theta$  must satisfy:

$$1 < \theta \leq 2$$

Selecting  $\theta = 2$  satisfies all conditions. Thus, the M-P neuron with weights  $w_1 = 1, w_2 = 1$  and threshold  $\theta = 2$  implements the AND gate.

### Hebb Net Weight Calculation

**Problem:** Train a Hebb net for the AND gate using bipolar inputs and targets.

**Solution:** The training patterns are:  $S^1 = (-1, -1), t^1 = -1$ ;  $S^2 = (1, -1), t^2 = -1$ ;  $S^3 = (-1, 1), t^3 = -1$ ;  $S^4 = (1, 1), t^4 = 1$ .

Initialize  $w_1 = 0, w_2 = 0, b = 0$ . The weight update rules are:  $\Delta w_1 = x_1 y, \Delta w_2 = x_2 y, \Delta b = y$ .

| Step | $x_1$ | $x_2$ | $y$ | $\Delta w_1$ | $\Delta w_2$ | $w_1$ | $w_2$ | $b$ |
|------|-------|-------|-----|--------------|--------------|-------|-------|-----|
| Init | -     | -     | -   | -            | -            | 0     | 0     | 0   |
| 1    | -1    | -1    | -1  | 1            | 1            | 1     | 1     | -1  |
| 2    | 1     | -1    | -1  | -1           | 1            | 0     | 2     | -2  |
| 3    | -1    | 1     | -1  | 1            | -1           | 1     | 1     | -3  |
| 4    | 1     | 1     | 1   | 1            | 1            | 2     | 2     | -2  |

The final weights are  $w_1 = 2, w_2 = 2$  and bias  $b = -2$ . The decision boundary is:  $2x_1 + 2x_2 - 2 = 0 \Rightarrow x_1 + x_2 = 1$ .

### BAM Correlation Matrix Computation

**Problem:** Store the bipolar hetero-associative pattern pair  $S = (1, -1, 1)$  and  $T = (1, -1)$  in a BAM. Calculate the weight matrix  $W$  and verify recall.

**Solution:** Given  $S = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix}$  and  $T = \begin{bmatrix} 1 & -1 \end{bmatrix}$ .

The weight matrix is  $W = S^T T$ :

$$W = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$$

**Verification of Forward Recall:** Given input  $S = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix}$ :

$$SW = \begin{bmatrix} 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 1(1) + (-1)(-1) + 1(1) & 1(-1) + (-1)(1) + 1(-1) \end{bmatrix}$$

$$SW = \begin{bmatrix} 3 & -3 \end{bmatrix}$$

Applying the sign function:

$$Y = \text{sign}(SW) = \begin{bmatrix} \text{sign}(3) & \text{sign}(-3) \end{bmatrix} = \begin{bmatrix} 1 & -1 \end{bmatrix} = T \quad (\text{Verified})$$

## Quick Revision — Module I

Neural Networks and Pattern Association | PE-EC702C

### Key Formulas at a Glance

|                                 |                                                                                                                   |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>M-P Neuron Output</b>        | $y = f(\sum w_i x_i) = \begin{cases} 1 & \text{if } \sum w_i x_i \geq \theta \\ 0 & \text{else} \end{cases}$      |
| <b>Hebbian Learning</b>         | $w_i(\text{new}) = w_i(\text{old}) + x_i y, \quad b(\text{new}) = b(\text{old}) + y$                              |
| <b>Perceptron Learning Rule</b> | $w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y)x_i, \quad b(\text{new}) = b(\text{old}) + \alpha(t - y)$       |
| <b>Adaline (LMS) Delta Rule</b> | $w_i(\text{new}) = w_i(\text{old}) + \alpha(t - \text{net})x_i \quad \text{where } \text{net} = \sum w_i x_i + b$ |
| <b>BAM Weight Matrix</b>        | $W = \sum_{p=1}^P (S^p)^T T^p$                                                                                    |
| <b>BAM Bidirectional Recall</b> | $Y = \text{sign}(XW), \quad X = \text{sign}(YW^T)$                                                                |

### One-Line Definitions

- **Activation Function:** A mathematical function that determines the neuron's output state based on its net input.
- **Linear Separability:** The property where data classes can be separated by a single hyperplane.
- **Hebbian Learning:** A learning paradigm where connection weights increase when pre- and post-synaptic units are active together.
- **Adaline:** A single-layer neural network trained using gradient descent to minimize mean squared output error.
- **BAM:** A hetero-associative recurrent neural network that stores bipolar vector pairs and performs bidirectional recall.

### Frequently Asked / Viva Questions

1. **Compare biological and artificial neural networks in terms of speed and processing style.**  
Biological networks process information in a massively parallel, slow ( $\sim$ ms) style. ANNs process sequentially/quasi-parallelly but at extremely high electronic speeds ( $\sim$ ns).
2. **Explain the XOR problem in single-layer perceptrons.**  
Single-layer perceptrons can only classify linearly separable patterns. XOR inputs cannot be separated by a single decision boundary line, rendering them unclassifiable by single-layer perceptrons.
3. **What is the significance of the bias term in a neuron?**

**Frequently Asked / Viva Questions (contd.)**

The bias acts as a shift factor for the activation function threshold, allowing the decision boundary to move away from the origin.

**4. What is the difference between Perceptron learning and Adaline learning?**

Perceptron learning updates weights based on the thresholded binary output, whereas Adaline learning uses the continuous pre-activation sum (Delta rule).

**5. Define the stability-plasticity dilemma.**

The dilemma of a network being stable enough to retain old learned patterns (stability) while remaining adaptive enough to learn new patterns (plasticity).

**6. Why is bipolar representation preferred over binary in Hebb nets?**

In binary representation (0, 1), inactive inputs (0) do not trigger weight updates, whereas bipolar representation (−1, 1) allows learning from negative states.

**7. How does BAM retrieve stored patterns bidirectionally?**

Inputs are multiplied by weight matrix  $W$  to yield outputs, which are then multiplied by transpose matrix  $W^T$  to retrieve inputs iteratively until stable convergence.

**8. What is the difference between supervised and unsupervised learning?**

Supervised learning uses target outputs to adjust weights based on error feedback. Unsupervised learning groups data patterns based on similarity metrics without targets.

**9. What is the role of inhibitory inputs in the McCulloch-Pitts neuron?**

Inhibitory inputs have absolute veto power: if an inhibitory input is active, the neuron output is forced to 0 regardless of excitatory inputs.

**10. State the convergence theorem of perceptrons.**

If the training data is linearly separable, the Perceptron learning algorithm is guaranteed to converge to a correct weight vector in a finite number of steps.

**Comparison of Learning Rules**

| Parameter               | Hebbian Learning                 | Perceptron Learning                          |
|-------------------------|----------------------------------|----------------------------------------------|
| Learning Type           | Unsupervised (self-association)  | Supervised (target-based)                    |
| Weight Update Condition | Always updates based on activity | Updates only on classification error         |
| Convergence Guarantee   | No guarantee of convergence      | Guaranteed convergence if linearly separable |
| Feedback Loop           | None                             | Error feedback loop present                  |

# Module 2

## Introduction to Competitive Learning

### 1. Introduction to Competitive Learning

Unlike feedforward networks that update weights based on output error signals, competitive learning networks are based on a winner-take-all strategy where nodes compete with each other to become active.

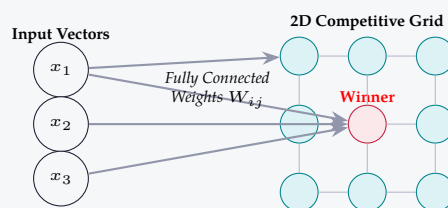
#### Competitive Learning

**Competitive Learning** is an unsupervised learning process where output neurons compete for activation. Only one neuron (the winning neuron) remains active at a time, and its weights are updated to become more similar to the input vector.

### 2. Kohonen Self-Organizing Maps (SOM)

Introduced by Teuvo Kohonen in 1982, the Self-Organizing Map (SOM) is an unsupervised learning network that maps high-dimensional input patterns to a low-dimensional (typically 1D or 2D) topological grid of neurons.

#### Kohonen Self-Organizing Map Architecture



#### 2.1 Kohonen SOM Learning Algorithm

The training process involves four major steps:

1. **Initialization:** Initialize weight vectors  $W_j = (w_{j1}, w_{j2}, \dots, w_{jn})^T$  for each grid node  $j$  with small random values. Set parameters: initial learning rate  $\alpha_0$  and initial neighborhood radius  $\sigma_0$ .
2. **Similarity Matching (Finding the BMU):** For a given input vector  $X = (x_1, x_2, \dots, x_n)^T$ ,

compute the Euclidean distance to all weight vectors:

$$d_j(X) = \|X - W_j\| = \sqrt{\sum_{i=1}^n (x_i - w_{ji})^2} \quad (13)$$

Identify the winning neuron (Best Matching Unit, BMU) index  $c$ :

$$c = \arg \min_j d_j(X) \quad (14)$$

3. **Weight Updating:** Update weights of the BMU and all neurons within its neighborhood  $h_{cj}(t)$ :

$$W_j(t+1) = W_j(t) + \alpha(t)h_{cj}(t)[X - W_j(t)] \quad (15)$$

4. **Decay Parameters:** Decay the learning rate and the neighborhood size:

$$\alpha(t) = \alpha_0 \exp\left(-\frac{t}{\tau_1}\right), \quad \sigma(t) = \sigma_0 \exp\left(-\frac{t}{\tau_2}\right) \quad (16)$$

### 3. Learning Vector Quantization (LVQ)

Learning Vector Quantization (LVQ), introduced by Kohonen, is a supervised competitive learning algorithm used for pattern classification.

#### LVQ Architecture

An **LVQ network** consists of an input layer and a competitive layer containing codebook vectors (prototype vectors). Each codebook vector is assigned a specific target class.

#### 3.1 LVQ Update Rules

Let  $w_c$  be the codebook vector closest to the input vector  $x$  (in terms of Euclidean distance). The weights are adjusted as follows:

#### LVQ Supervised Training updates

If the winning codebook vector  $w_c$  belongs to the **correct class** as the input  $x$ :

$$w_c(t+1) = w_c(t) + \alpha(t)[x - w_c(t)] \quad (17)$$

If the winning codebook vector  $w_c$  belongs to an **incorrect class** relative to input  $x$ :

$$w_c(t+1) = w_c(t) - \alpha(t)[x - w_c(t)] \quad (18)$$

All other codebook vectors remain unchanged:

$$w_j(t+1) = w_j(t) \quad \text{for } j \neq c \quad (19)$$

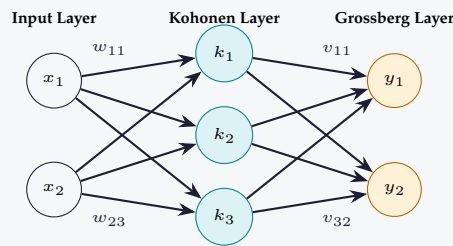
### 4. Counter Propagation Networks (CPN)

Developed by Robert Hecht-Nielsen in 1987, CPN combines a competitive unsupervised layer (Kohonen SOM) with a self-organizing outstar layer (Grossberg outstar). It acts as a fast lookup table to approximate continuous mathematical functions.

#### 4.1 CPN Architecture

CPN contains three layers: Input layer, Kohonen layer (competitive), and Grossberg (Outstar) output layer.

##### Counter Propagation Network (CPN) Layout



#### 4.2 CPN Training phases

1. **First Phase (Kohonen Training):** The input vector  $X$  is presented. The Kohonen layer acts as a competitive classifier. The winning neuron  $c$  is determined, and its weights  $W_c$  are updated using standard Kohonen rules.
2. **Second Phase (Grossberg Training):** The Kohonen layer outputs a binary vector where only the winning unit has activation 1, and all others are 0 ( $z_j = \delta_{jc}$ ). The Grossberg weights  $V$  are updated to match the target vector  $Y$ :

$$v_{jk}(t+1) = v_{jk}(t) + \beta[y_k - v_{jk}(t)]z_j \quad (20)$$

For the winner  $c$ :  $V_c(t+1) = V_c(t) + \beta[Y - V_c(t)]$ .

### 5. Worked Examples

#### Kohonen SOM Winning Neuron and Weight Update

**Problem:** Given an input vector  $X = \begin{bmatrix} 0.8 & 0.6 \end{bmatrix}^T$  and three Kohonen neurons with weight vectors:

$$W_1 = \begin{bmatrix} 0.9 & 0.1 \end{bmatrix}^T, \quad W_2 = \begin{bmatrix} 0.7 & 0.5 \end{bmatrix}^T, \quad W_3 = \begin{bmatrix} 0.2 & 0.8 \end{bmatrix}^T$$

- (a) Identify the winning neuron (BMU) using Euclidean distance.
- (b) Update the weight vector of the winning neuron using learning rate  $\alpha = 0.5$ .

**Solution:**

**(a) Similarity Matching:** Compute the squared Euclidean distance  $D_j = ||X - W_j||^2$  for each neuron:

- For  $W_1$ :

$$D_1 = (0.8 - 0.9)^2 + (0.6 - 0.1)^2 = (-0.1)^2 + (0.5)^2 = 0.01 + 0.25 = 0.26$$

- For  $W_2$ :

$$D_2 = (0.8 - 0.7)^2 + (0.6 - 0.5)^2 = (0.1)^2 + (0.1)^2 = 0.01 + 0.01 = 0.02$$

- For  $W_3$ :

$$D_3 = (0.8 - 0.2)^2 + (0.6 - 0.8)^2 = (0.6)^2 + (-0.2)^2 = 0.36 + 0.04 = 0.40$$

Since  $D_2 = 0.02$  is the minimum, **Neuron 2** ( $W_2$ ) is the winning neuron (BMU).

**(b) Weight Update:** Update the weights of the winning neuron  $W_2$ :

$$\begin{aligned} W_2(\text{new}) &= W_2(\text{old}) + \alpha[X - W_2(\text{old})] \\ &= \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix} + 0.5 \left( \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix} - \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix} \right) \\ &= \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix} + 0.5 \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.05 \\ 0.05 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.55 \end{bmatrix} \end{aligned}$$

The updated weight vector is  $W_2 = \begin{bmatrix} 0.75 & 0.55 \end{bmatrix}^T$ .

### LVQ Classifier Updates

**Problem:** Given an input vector  $x = \begin{bmatrix} 0.1 & 0.9 \end{bmatrix}$  belonging to Class 1, and the closest codebook vector  $w_c = \begin{bmatrix} 0.2 & 0.7 \end{bmatrix}$ . Calculate the updated codebook vector for:

- Class of  $w_c$  is Class 1 (correct class)
- Class of  $w_c$  is Class 2 (incorrect class)

Assume learning rate  $\alpha = 0.2$ .

---

**Solution:**

**(a) Correct Class Update (Push towards input):**

$$\begin{aligned}
 w_c(new) &= w_c(old) + \alpha[x - w_c(old)] \\
 &= \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} + 0.2 \left( \begin{bmatrix} 0.1 \\ 0.9 \end{bmatrix} - \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} \right) \\
 &= \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} + 0.2 \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.2 - 0.02 \\ 0.7 + 0.04 \end{bmatrix} = \begin{bmatrix} 0.18 \\ 0.74 \end{bmatrix}
 \end{aligned}$$

**(b) Incorrect Class Update (Pull away from input):**

$$\begin{aligned}
 w_c(new) &= w_c(old) - \alpha[x - w_c(old)] \\
 &= \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} - 0.2 \left( \begin{bmatrix} 0.1 \\ 0.9 \end{bmatrix} - \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} \right) \\
 &= \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} - 0.2 \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.2 + 0.02 \\ 0.7 - 0.04 \end{bmatrix} = \begin{bmatrix} 0.22 \\ 0.66 \end{bmatrix}
 \end{aligned}$$

## Quick Revision — Module II

Competitive Neural Networks | PE-EC702C

### Key Formulas at a Glance

|                                   |                                                      |
|-----------------------------------|------------------------------------------------------|
| <b>Euclidean Distance in SOM</b>  | $d_j(X) = \sqrt{\sum_{i=1}^n (x_i - w_{ji})^2}$      |
| <b>Winning Neuron (BMU) Index</b> | $c = \arg \min_j d_j(X)$                             |
| <b>Kohonen Weight Update</b>      | $W_j(t+1) = W_j(t) + \alpha(t)h_{cj}(t)[X - W_j(t)]$ |
| <b>LVQ Correct Update</b>         | $w_c(t+1) = w_c(t) + \alpha(t)[x - w_c(t)]$          |
| <b>LVQ Incorrect Update</b>       | $w_c(t+1) = w_c(t) - \alpha(t)[x - w_c(t)]$          |
| <b>Grossberg outstar Update</b>   | $V_c(t+1) = V_c(t) + \beta[Y - V_c(t)]$              |

### One-Line Definitions

- **Kohonen SOM:** An unsupervised clustering network that creates a topology-preserving mapping of high-dimensional input features.
- **Best Matching Unit (BMU):** The neuron in a competitive layer whose weight vector is closest to the current input vector.
- **LVQ:** A supervised competitive network that uses prototype vectors to map inputs to specific classification boundaries.
- **CPN:** A hybrid network that maps input patterns to target vectors rapidly by combining a Kohonen layer with a Grossberg layer.
- **Grossberg Outstar:** A learning unit designed to output a specific analog target pattern when triggered by a Kohonen node.

### Frequently Asked / Viva Questions

1. **What is the primary difference between Perceptron learning and Kohonen learning?**  
Perceptron learning is supervised and minimizes output error feedback. Kohonen learning is unsupervised and uses similarity metrics to cluster features.
2. **How does the neighborhood function  $h_{cj}(t)$  change during SOM training?**  
It starts wide to organize the global structure of the map, and then shrinks exponentially over time to allow fine-tuning of local weights.
3. **Why is LVQ considered a supervised learning algorithm while SOM is unsupervised?**  
SOM groups data based solely on input similarities without class labels. LVQ adjusts its codebook vectors based on class labels to optimize classification boundaries.
4. **Explain the role of Grossberg outstar layer in Counter Propagation Networks.**

**Frequently Asked / Viva Questions (contd.)**

The Kohonen layer maps inputs to a single active winning unit. The Grossberg layer acts as a lookup table, outputting a stored target vector associated with that unit.

**5. What is the significance of the vigilance parameter in competitive networks?**

It controls the sensitivity of node matching. In networks like ART, it determines if a new cluster must be created or if the input fits an existing cluster.

**6. Why does CPN train faster than Backpropagation networks?**

CPN updates only the winning Kohonen neuron and the corresponding Grossberg weights, avoiding expensive error propagation across multiple layers.

**7. What is the purpose of lateral inhibition in competitive networks?**

It allows active neurons to suppress the activation of neighboring competitive neurons, ensuring that only one winning neuron (the BMU) remains active.

**8. How does LVQ-2 differ from standard LVQ-1?**

LVQ-1 updates only the single closest codebook vector. LVQ-2 updates the two closest vectors simultaneously if the input falls near the decision boundary.

**9. What is meant by a topology-preserving mapping in SOM?**

It means that input vectors that are close in the high-dimensional feature space are mapped to neurons that are close to each other on the 2D output grid.

**10. What is the codebook vector in Learning Vector Quantization?**

It is a prototype vector representing a class cluster center in the input space.

**Comparison of Competitive Networks**

| Attribute           | Kohonen SOM                | LVQ                    | CPN                                |
|---------------------|----------------------------|------------------------|------------------------------------|
| Learning Type       | Unsupervised               | Supervised             | Hybrid (Supervised + Unsupervised) |
| Primary Application | Dimensionality reduction   | Pattern classification | Function approximation             |
| Output Layer Grid   | 1D or 2D grid structure    | 1D array of vectors    | 1D array of output nodes           |
| Weight Updates      | Updates winner + neighbors | Updates winner only    | Updates winner only                |

# Module 3

## Backpropagation Neural Networks (BPNN)

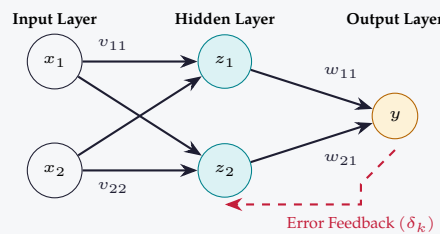
### 1. Backpropagation Neural Networks (BPNN)

The Backpropagation algorithm is the standard method for training Multi-Layer Perceptrons (MLPs). It is a supervised learning method based on the generalized delta rule (gradient descent on error space).

#### Multi-Layer Perceptron (MLP)

A **Multi-Layer Perceptron (MLP)** is a feedforward artificial neural network topology consisting of an input layer, one or more hidden layers, and an output layer of non-linear processing units.

#### Multi-Layer Perceptron Feedforward Architecture



#### 1.1 Derivation of the Backpropagation Learning Rules

Let  $E$  be the total sum-of-squares error over the output units:

$$E = \frac{1}{2} \sum_k (t_k - y_k)^2 \quad (21)$$

where  $t_k$  is the target output and  $y_k$  is the computed activation of output unit  $k$ . The activation function used is the binary sigmoid:

$$y_k = f(net_k) = \frac{1}{1 + e^{-net_k}} \rightarrow f'(net_k) = y_k(1 - y_k) \quad (22)$$

where  $net_k = \sum_j w_{kj} z_j + b_k$  ( $z_j$  is the activation of hidden node  $j$ ).

Using gradient descent, the weight changes are proportional to the negative gradient of error:

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial w_{kj}} \quad (23)$$

By applying the chain rule of calculus:

$$\frac{\partial E}{\partial w_{kj}} = \frac{\partial E}{\partial y_k} \cdot \frac{\partial y_k}{\partial net_k} \cdot \frac{\partial net_k}{\partial w_{kj}} \quad (24)$$

Evaluating each derivative:

$$\frac{\partial E}{\partial y_k} = -(t_k - y_k), \quad \frac{\partial y_k}{\partial net_k} = f'(net_k) = y_k(1 - y_k), \quad \frac{\partial net_k}{\partial w_{kj}} = z_j$$

Substituting these terms back:

$$\frac{\partial E}{\partial w_{kj}} = -(t_k - y_k) f'(net_k) z_j \quad (25)$$

Defining the error term (delta) for output unit  $k$  as  $\delta_k = (t_k - y_k) f'(net_k)$ :

$$\Delta w_{kj} = \eta \delta_k z_j \quad (26)$$

For input-to-hidden weights  $v_{ji}$  connecting input  $i$  to hidden unit  $j$ :

$$\Delta v_{ji} = -\eta \frac{\partial E}{\partial v_{ji}} = -\eta \frac{\partial E}{\partial z_j} \cdot \frac{\partial z_j}{\partial net_j} \cdot \frac{\partial net_j}{\partial v_{ji}} \quad (27)$$

where  $z_j = f(net_j)$  and  $net_j = \sum_i v_{ji} x_i + b_j$ . The error gradient relative to hidden activation  $z_j$  accumulates from all output nodes:

$$\frac{\partial E}{\partial z_j} = \sum_k \frac{\partial E}{\partial net_k} \cdot \frac{\partial net_k}{\partial z_j} = \sum_k (-\delta_k) w_{kj}$$

Applying the other terms:

$$\frac{\partial z_j}{\partial net_j} = f'(net_j) = z_j(1 - z_j), \quad \frac{\partial net_j}{\partial v_{ji}} = x_i$$

Defining the error term (delta) for hidden unit  $j$  as  $\delta_j = f'(net_j) \sum_k \delta_k w_{kj}$ :

$$\Delta v_{ji} = \eta \delta_j x_i \quad (28)$$

## 2. Adaptive Resonance Theory (ART)

Developed by Stephen Grossberg and Gail Carpenter (1987), ART is designed to solve the stability-plasticity dilemma in unsupervised neural networks.

### Stability-Plasticity Dilemma

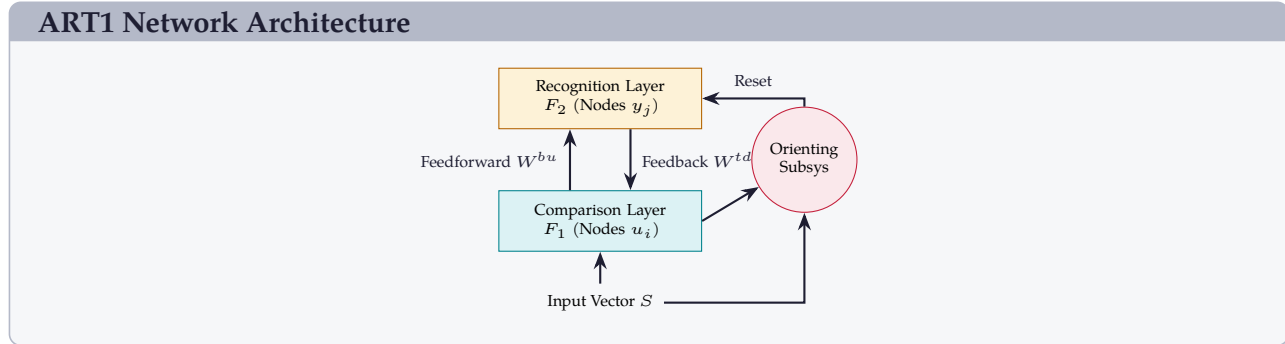
**Stability-Plasticity Dilemma:** The challenge of designing a learning system that remains adaptive to new patterns (plasticity) while preventing new learning from washing out previously acquired knowledge (stability).

## 2.1 ART1 vs. ART2

- **ART1:** Designed to process binary input vectors ( $\{0, 1\}$ ).
- **ART2:** Designed to process continuous (analog) input vectors.

## 2.2 ART1 Architecture and Operation

ART1 consists of two subsystems: the Attentional Subsystem and the Orienting Subsystem.



### Working Mechanism:

1. **Input presentation:** Input  $S$  is sent to the  $F_1$  layer.
2. **Feedforward activation:**  $F_1$  transmits the pattern to the  $F_2$  layer. The  $F_2$  layer is a competitive layer where the node with the highest net input wins (winner-take-all).
3. **Feedback Template matching:** The winning  $F_2$  neuron projects its prototype vector (template) back down to  $F_1$ .
4. **Vigilance Test:** The orienting subsystem calculates the ratio of match:

$$\frac{|S \cap X|}{|S|} \quad (29)$$

where  $X$  is the activation of the  $F_1$  layer.

- If ratio  $\geq \rho$  (vigilance parameter), **resonance** occurs. The network accepts the match and updates the winning neuron's weights.
- If ratio  $< \rho$ , the orienting subsystem sends a **reset signal** to temporarily disable the winning  $F_2$  node. The search process repeats for other  $F_2$  nodes. If no match is found, a new category node is initialized.

## 3. Boltzmann Machine Learning

The Boltzmann Machine is a recurrent, symmetrically connected network consisting of stochastic binary units (visible and hidden) that operate under statistical thermodynamics principles.

### Boltzmann Energy & Transition Rules

The total energy  $E$  of a Boltzmann state configuration is defined as:

$$E = - \sum_{i < j} w_{ij} s_i s_j - \sum_i \theta_i s_i \quad (30)$$

where  $w_{ij}$  is the connection weight,  $s_i \in \{0, 1\}$  is the state of unit  $i$ , and  $\theta_i$  is the threshold bias.

The probability that unit  $i$  transitions to state  $s_i = 1$  is given by the sigmoid-like logistic function:

$$P(s_i = 1) = \frac{1}{1 + e^{-\Delta E_i / T}} \quad (31)$$

where  $\Delta E_i = \sum_j w_{ij} s_j + \theta_i$  is the energy difference and  $T$  is the temperature parameter.

### 3.1 Boltzmann Learning Rule

Boltzmann training occurs in two distinct phases:

1. **Positive (Clamped) Phase:** The visible units are clamped to target training patterns. The network is allowed to reach thermal equilibrium, and the correlation probabilities  $p_{ij}^+$  that units  $i$  and  $j$  are active simultaneously are measured.
2. **Negative (Free-Running) Phase:** No units are clamped. The network runs freely to reach thermal equilibrium, and the free probabilities  $p_{ij}^-$  are measured.
3. **Weight Update:** Weights are updated to minimize Kullback-Leibler divergence:

$$\Delta w_{ij} = \eta (p_{ij}^+ - p_{ij}^-) \quad (32)$$

## 4. Worked Examples

### Backpropagation Step-by-Step Numerical

**Problem:** Consider a simplified neural network containing 1 input node ( $x$ ), 1 hidden node ( $z$ ), and 1 output node ( $y$ ). The transfer function is the binary sigmoid:  $f(net) = 1/(1 + e^{-net})$ .

Given: Input  $x = 1.0$ , target  $t = 0.9$ , learning rate  $\eta = 0.5$ . Initial weights and biases:

- Input-to-hidden weight  $v = 0.5$ , bias  $b_1 = 0.2$
- Hidden-to-output weight  $w = 0.8$ , bias  $b_2 = 0.1$

Calculate the updated weights  $w$  and  $v$  after a single backpropagation epoch.

**Solution:**

1. **Forward Pass:**

- **Hidden Node  $z$ :**

$$net_z = v \cdot x + b_1 = 0.5(1.0) + 0.2 = 0.7$$

$$z = f(net_z) = \frac{1}{1 + e^{-0.7}} \approx 0.668$$

- **Output Node  $y$ :**

$$net_y = w \cdot z + b_2 = 0.8(0.668) + 0.1 \approx 0.634$$

$$y = f(net_y) = \frac{1}{1 + e^{-0.634}} \approx 0.653$$

## 2. Backward Pass (Error Propagation):

- **Output Delta ( $\delta_y$ ):**

$$\begin{aligned}\delta_y &= (t - y)y(1 - y) \\ &= (0.9 - 0.653) \cdot 0.653 \cdot (1 - 0.653) \\ &= 0.247 \cdot 0.653 \cdot 0.347 \approx 0.0560\end{aligned}$$

- **Hidden Delta ( $\delta_z$ ):**

$$\begin{aligned}\delta_z &= \delta_y \cdot w \cdot z(1 - z) \\ &= 0.0560 \cdot 0.8 \cdot 0.668 \cdot (1 - 0.668) \\ &= 0.0448 \cdot 0.668 \cdot 0.332 \approx 0.00993\end{aligned}$$

## 3. Weight Updates:

- **Hidden-to-Output Weight ( $w$ ):**

$$\begin{aligned}\Delta w &= \eta \cdot \delta_y \cdot z = 0.5 \cdot 0.0560 \cdot 0.668 \approx 0.0187 \\ w(new) &= 0.8 + 0.0187 = 0.8187\end{aligned}$$

- **Input-to-Hidden Weight ( $v$ ):**

$$\begin{aligned}\Delta v &= \eta \cdot \delta_z \cdot x = 0.5 \cdot 0.00993 \cdot 1.0 \approx 0.00497 \\ v(new) &= 0.5 + 0.00497 = 0.50497\end{aligned}$$

The updated weights are  $w \approx 0.8187$  and  $v \approx 0.5050$ .

## Quick Revision — Module III

Adaptive Resonance & Backpropagation | PE-EC702C

### Key Formulas at a Glance

|                                                   |                                                                              |
|---------------------------------------------------|------------------------------------------------------------------------------|
| <b>Total Output Error</b>                         | $E = \frac{1}{2} \sum_k (t_k - y_k)^2$                                       |
| <b>Output Layer Delta (<math>\delta_k</math>)</b> | $\delta_k = (t_k - y_k)y_k(1 - y_k)$ (for sigmoid)                           |
| <b>Hidden Layer Delta (<math>\delta_j</math>)</b> | $\delta_j = z_j(1 - z_j) \sum_k \delta_k w_{kj}$ (for sigmoid)               |
| <b>BPNN Weight Updates</b>                        | $\Delta w_{kj} = \eta \delta_k z_j, \quad \Delta v_{ji} = \eta \delta_j x_i$ |
| <b>ART1 Vigilance Ratio</b>                       | $\text{Match} = \frac{ S \cap X }{ S } \geq \rho$ (where $\rho \in [0, 1]$ ) |
| <b>Boltzmann Energy Configuration</b>             | $E = - \sum_{i < j} w_{ij} s_i s_j - \sum_i \theta_i s_i$                    |

### One-Line Definitions

- **Backpropagation:** A supervised gradient-descent algorithm that updates network weights to minimize the sum of squared errors.
- **Stability-Plasticity Dilemma:** The goal of learning new input patterns without overwriting previously learned representations.
- **Vigilance Parameter ( $\rho$ ):** The threshold parameter in ART that determines how closely an input must match a category template.
- **Resonance:** The state in ART where an input matches a template well enough to initiate weight tuning.
- **Boltzmann Machine:** A recurrent stochastic neural network that uses simulated annealing to escape local energy minima.

### Frequently Asked / Viva Questions

1. **Why is Backpropagation referred to as a "supervised" learning algorithm?**  
Because weight adjustments are calculated using the difference (error) between the network's computed output and the target (label) vector.
2. **State the purpose of the activation function's derivative  $f'(net)$  in backpropagation.**  
It scales the weight update based on the slope of the activation curve. In flat regions (near 0 or 1 for sigmoids), updates are very small, preventing oscillations.
3. **What is the effect of setting a high vigilance parameter ( $\rho$ ) in ART1?**  
A high vigilance value (e.g.  $\rho = 0.95$ ) requires highly precise matches, leading the network to create many narrow, fine-grained categories.
4. **What is the effect of setting a low vigilance parameter ( $\rho$ ) in ART1?**

## Frequently Asked / Viva Questions (contd.)

A low vigilance value (e.g.  $\rho = 0.3$ ) allows loose matching, causing the network to group highly different inputs into a few broad, coarse categories.

5. **How does Boltzmann learning utilize the positive and negative phases?**

The positive phase clamps inputs to capture target correlations ( $p_{ij}^+$ ), while the negative phase runs freely to capture network-generated correlations ( $p_{ij}^-$ ).

6. **Why does backpropagation sometimes get stuck in local minima?**

Because gradient descent follows local gradients on the error surface. If it enters a local valley (minimum) that is not the absolute lowest point (global minimum), it cannot escape.

7. **What is the function of the orienting subsystem in ART1?**

It acts as a controller that compares the input match ratio with the vigilance parameter  $\rho$ . If it is too low, it issues a reset signal to disable the winning category node.

8. **Define the term "Simulated Annealing" in Boltzmann machines.**

The process of starting at a high temperature  $T$  to allow stochastic state jumps (escaping local minima), and slowly cooling down to settle in the global minimum state.

9. **What is the difference between feedforward and feedback networks in terms of stability?**

Feedforward networks compute outputs instantaneously and are always stable. Feedback networks have recurrent loop dynamics and can oscillate or require iterative convergence to reach stability.

10. **Why is  $\frac{1}{2}$  added in the error formulation  $E = \frac{1}{2} \sum (t - y)^2$ ?**

It is a mathematical convenience that cancels out the coefficient 2 when taking the derivative  $\frac{\partial E}{\partial y} = -(t - y)$  during gradient calculations.

## Comparison of Adaptive Architectures

| Attribute       | Backpropagation BPNN | ART1                               | Boltzmann Machine                |
|-----------------|----------------------|------------------------------------|----------------------------------|
| Learning Type   | Supervised           | Unsupervised                       | Supervised / Unsupervised        |
| Connections     | Feedforward only     | Bidirectional (top-down/bottom-up) | Symmetric recurrent loops        |
| Neuron States   | Analog / Continuous  | Binary $\{0, 1\}$                  | Stochastic Binary $\{0, 1\}$     |
| Page Resolution | Fixed classes        | Dynamic category creation          | Probability distribution fitting |

# Module 4

## Classical (Crisp) Sets vs. Fuzzy Sets

### 1. Classical (Crisp) Sets vs. Fuzzy Sets

Classical set theory is based on binary logic (true/false, black/white), whereas fuzzy set theory allows for gradual transitions and degrees of membership.

#### Fuzzy Set

A **Fuzzy Set**  $A$  in a universe of discourse  $U$  is defined by a membership function  $\mu_A(x)$  that maps each element  $x \in U$  to a continuous value in the interval  $[0, 1]$ .

$$A = \{(x, \mu_A(x)) \mid x \in U\} \quad (33)$$

#### 1.1 Standard Fuzzy Operations

Let  $A$  and  $B$  be two fuzzy sets with membership functions  $\mu_A(x)$  and  $\mu_B(x)$  respectively:

- **Union (OR / Standard Max):**

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)) = \mu_A(x) \vee \mu_B(x) \quad (34)$$

- **Intersection (AND / Standard Min):**

$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)) = \mu_A(x) \wedge \mu_B(x) \quad (35)$$

- **Complement (NOT):**

$$\mu_{\bar{A}}(x) = 1 - \mu_A(x) \quad (36)$$

- **Algebraic Product (Dot):**  $\mu_{A \cdot B}(x) = \mu_A(x) \cdot \mu_B(x)$

- **Algebraic Sum (Probabilistic sum):**  $\mu_{A \hat{+} B}(x) = \mu_A(x) + \mu_B(x) - \mu_A(x)\mu_B(x)$

### 2. Fuzzy Relations and Composition

Fuzzy relations map elements from universe  $X$  to universe  $Y$  with a membership value in  $[0, 1]$ .

### Fuzzy Compositions

Let  $R$  be a fuzzy relation from  $X$  to  $Y$ , and  $S$  be a fuzzy relation from  $Y$  to  $Z$ .

#### 1. Max-Min Composition ( $T = R \circ S$ ):

$$\mu_T(x, z) = \max_{y \in Y} \{ \min(\mu_R(x, y), \mu_S(y, z)) \} \quad (37)$$

#### 2. Max-Product Composition ( $T = R \bullet S$ ):

$$\mu_T(x, z) = \max_{y \in Y} \{ \mu_R(x, y) \cdot \mu_S(y, z) \} \quad (38)$$

## 2.1 Tolerance and Equivalence Relations

Fuzzy relations can possess algebraic properties similar to classical relations:

- **Reflexivity:** A relation  $R$  on set  $X$  is reflexive if the membership of any element paired with itself is 1:

$$\mu_R(x, x) = 1 \quad \forall x \in X \quad (39)$$

- **Symmetry:** A relation  $R$  is symmetric if order of elements does not affect membership:

$$\mu_R(x, y) = \mu_R(y, x) \quad \forall x, y \in X \quad (40)$$

- **Transitivity (Max-Min Transitivity):** A relation  $R$  is Max-Min transitive if:

$$R \circ R \subseteq R \quad \implies \quad \mu_R(x, z) \geq \max_{y \in X} \{ \min(\mu_R(x, y), \mu_R(y, z)) \} \quad \forall x, z \in X \quad (41)$$

Based on these properties, we define two special fuzzy relations:

1. **Tolerance Relation (Compatibility Relation):** A fuzzy relation that is **reflexive** and **symmetric**, but not necessarily transitive. It measures the degree of similarity or compatibility between elements.
2. **Equivalence Relation (Similarity Relation):** A fuzzy relation that is **reflexive**, **symmetric**, and **Max-Min transitive**. It generalizes the classical equivalence relation to group elements into fuzzy equivalence classes.

## 3. Membership Functions and Standard Forms

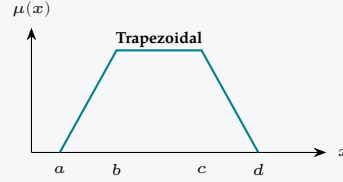
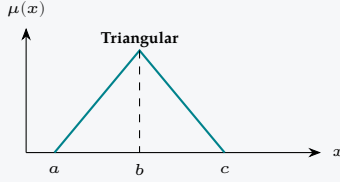
### 3.1 Key Geometric Features

- **Support:** The crisp set containing all elements with non-zero membership:  $\{x \in U \mid \mu_A(x) > 0\}$ .
- **Core:** The crisp set containing all elements with full membership:  $\{x \in U \mid \mu_A(x) = 1\}$ .
- **Boundary:** The region with  $0 < \mu_A(x) < 1$ .
- **Crossover Points:** The elements where membership is exactly 0.5:  $\{x \in U \mid \mu_A(x) = 0.5\}$ .

### 3.2 Standard Shapes

Standard shapes are used to represent categories like "Warm Temperature" or "High Voltage":

#### Standard Membership Function Shapes



## 4. Fuzzification and Defuzzification

- **Fuzzification:** The process of converting a crisp physical input value into a fuzzy membership value using predefined membership functions.
- **Defuzzification:** The process of converting a combined fuzzy output shape into a single representative crisp output value.

### 4.1 Standard Defuzzification Methods

Let  $C$  be the output fuzzy set defined on the continuous output space  $z$ .

#### Mathematical Formulations of Defuzzification

##### 1. Centroid (Center of Gravity - COG):

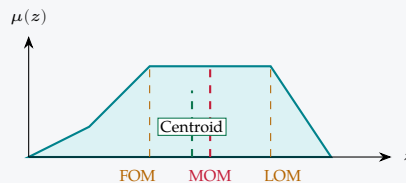
$$z^* = \frac{\int z \mu_C(z) dz}{\int \mu_C(z) dz} \rightarrow \text{(Discrete: } z^* = \frac{\sum z_i \mu_C(z_i)}{\sum \mu_C(z_i)}) \quad (42)$$

##### 2. Mean of Maxima (MOM):

$$z^* = \frac{\sum_{z_i \in M} z_i}{|M|} \quad \text{where } M = \{z \mid \mu_C(z) \text{ is maximum}\} \quad (43)$$

##### 3. First of Maxima (FOM) / Last of Maxima (LOM): $z^* = \min(M)$ (First) or $z^* = \max(M)$ (Last).

#### Defuzzification Geometric Points



## 5. Lambda Cuts ( $\lambda$ -cuts)

A  $\lambda$ -cut (or  $\alpha$ -cut) operation converts a fuzzy set or fuzzy relation into a classical (crisp) set by thresholding.

### Lambda Cut ( $\lambda$ -cut)

The  $\lambda$ -cut of a fuzzy set  $A$ , denoted by  $A_\lambda$ , is a crisp set containing all elements whose membership values are greater than or equal to  $\lambda$ :

$$A_\lambda = \{x \in U \mid \mu_A(x) \geq \lambda\} \quad \text{where } \lambda \in [0, 1] \quad (44)$$

## 6. Worked Examples

### Fuzzy Set Operations

**Problem:** Given two discrete fuzzy sets defined on  $U = \{1, 2, 3, 4\}$ :

$$A = \left\{ \frac{0.2}{1}, \frac{0.7}{2}, \frac{1.0}{3}, \frac{0.4}{4} \right\}, \quad B = \left\{ \frac{0.5}{1}, \frac{0.6}{2}, \frac{0.3}{3}, \frac{0.9}{4} \right\}$$

Calculate: (a)  $A \cup B$ , (b)  $A \cap B$ , and (c) the algebraic product  $A \cdot B$ .

**Solution:** Recall:  $\mu_{A \cup B} = \max(\mu_A, \mu_B)$ ,  $\mu_{A \cap B} = \min(\mu_A, \mu_B)$ , and  $\mu_{A \cdot B} = \mu_A \cdot \mu_B$ .

• (a) **Union ( $A \cup B$ ):**

$$\mu_{A \cup B}(1) = \max(0.2, 0.5) = 0.5$$

$$\mu_{A \cup B}(2) = \max(0.7, 0.6) = 0.7$$

$$\mu_{A \cup B}(3) = \max(1.0, 0.3) = 1.0$$

$$\mu_{A \cup B}(4) = \max(0.4, 0.9) = 0.9$$

$$A \cup B = \left\{ \frac{0.5}{1}, \frac{0.7}{2}, \frac{1.0}{3}, \frac{0.9}{4} \right\}$$

- (b) Intersection ( $A \cap B$ ):

$$\begin{aligned} A \cap B &= \left\{ \frac{\min(0.2, 0.5)}{1}, \frac{\min(0.7, 0.6)}{2}, \frac{\min(1.0, 0.3)}{3}, \frac{\min(0.4, 0.9)}{4} \right\} \\ &= \left\{ \frac{0.2}{1}, \frac{0.6}{2}, \frac{0.3}{3}, \frac{0.4}{4} \right\} \end{aligned}$$

- (c) Algebraic Product ( $A \cdot B$ ):

$$\begin{aligned} A \cdot B &= \left\{ \frac{0.2 \times 0.5}{1}, \frac{0.7 \times 0.6}{2}, \frac{1.0 \times 0.3}{3}, \frac{0.4 \times 0.9}{4} \right\} \\ &= \left\{ \frac{0.10}{1}, \frac{0.42}{2}, \frac{0.30}{3}, \frac{0.36}{4} \right\} \end{aligned}$$

### Max-Min Composition of Fuzzy Relations

**Problem:** Given fuzzy relations  $R$  (size  $2 \times 2$ ) and  $S$  (size  $2 \times 2$ ):

$$R = \begin{bmatrix} 0.6 & 0.3 \\ 0.2 & 0.9 \end{bmatrix}, \quad S = \begin{bmatrix} 0.5 & 0.8 \\ 0.7 & 0.1 \end{bmatrix}$$

Compute the Max-Min composition  $T = R \circ S$ .

**Solution:** The formulation is  $t_{ik} = \max_j \{ \min(r_{ij}, s_{jk}) \}$ .

- Element  $t_{11}$ :

$$t_{11} = \max(\min(0.6, 0.5), \min(0.3, 0.7)) = \max(0.5, 0.3) = 0.5$$

- Element  $t_{12}$ :

$$t_{12} = \max(\min(0.6, 0.8), \min(0.3, 0.1)) = \max(0.6, 0.1) = 0.6$$

- Element  $t_{21}$ :

$$t_{21} = \max(\min(0.2, 0.5), \min(0.9, 0.7)) = \max(0.2, 0.7) = 0.7$$

- Element  $t_{22}$ :

$$t_{22} = \max(\min(0.2, 0.8), \min(0.9, 0.1)) = \max(0.2, 0.1) = 0.2$$

Thus, the composition is:

$$T = R \circ S = \begin{bmatrix} 0.5 & 0.6 \\ 0.7 & 0.2 \end{bmatrix}$$

**Discrete Centroid Defuzzification**

**Problem:** Calculate the crisp value  $z^*$  of the fuzzy set  $C$  using Centroid defuzzification:

$$C = \left\{ \frac{0.1}{10}, \frac{0.3}{20}, \frac{0.8}{30}, \frac{1.0}{40}, \frac{0.6}{50} \right\}$$

**Solution:** We use the discrete Centroid formula:  $z^* = \frac{\sum z_i \mu_C(z_i)}{\sum \mu_C(z_i)}$ .

$$\begin{aligned} \sum z_i \mu_C(z_i) &= 10(0.1) + 20(0.3) + 30(0.8) + 40(1.0) + 50(0.6) \\ &= 1 + 6 + 24 + 40 + 30 = 101 \end{aligned}$$

$$\sum \mu_C(z_i) = 0.1 + 0.3 + 0.8 + 1.0 + 0.6 = 2.8$$

$$z^* = \frac{101}{2.8} \approx 36.07$$

The defuzzified crisp output is  $z^* \approx 36.07$ .

## Quick Revision — Module IV

Fuzzy Sets and Membership Functions | PE-EC702C

### Key Formulas at a Glance

|                                              |                                                                                                                       |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Fuzzy Union (OR)</b>                      | $\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$                                                                        |
| <b>Fuzzy Intersection (AND)</b>              | $\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$                                                                        |
| <b>Max-Min Composition</b>                   | $\mu_{R \circ S}(x, z) = \max_y \{ \min(\mu_R(x, y), \mu_S(y, z)) \}$                                                 |
| <b>Max-Product Composition</b>               | $\mu_{R \bullet S}(x, z) = \max_y \{ \mu_R(x, y) \cdot \mu_S(y, z) \}$                                                |
| <b>Centroid Defuzzification</b>              | $z^* = \frac{\sum z_i \mu_C(z_i)}{\sum \mu_C(z_i)} \quad \text{or} \quad \frac{\int z \mu_C(z) dz}{\int \mu_C(z) dz}$ |
| <b>Lambda-Cut (<math>\lambda</math>-cut)</b> | $A_\lambda = \{x \in U \mid \mu_A(x) \geq \lambda\}$                                                                  |

### One-Line Definitions

- **Fuzzy Set:** A set containing elements with varying degrees of membership represented by values in  $[0, 1]$ .
- **Membership Function:** The graphical curve or mathematical function that maps elements to their membership degrees.
- **Centroid Defuzzification:** A defuzzification method that calculates the center of gravity of the output fuzzy region.
- **Lambda Cut:** An operation that converts a fuzzy set/relation into a classical set using a threshold value  $\lambda$ .
- **Crossover Point:** Any element in the universe of discourse whose membership degree in a fuzzy set is exactly 0.5.

### Frequently Asked / Viva Questions

#### 1. What is the difference between probability and fuzziness?

Probability deals with the likelihood/chance of an event occurring (undecided outcome). Fuzziness deals with the vagueness of an event that has already occurred (deterministic but vague boundaries, e.g. "He is tall").

#### 2. State the properties of classical sets that do NOT hold in fuzzy sets.

The Law of Contradiction ( $A \cap \bar{A} = \emptyset$ ) and the Law of Excluded Middle ( $A \cup \bar{A} = U$ ) do not hold in fuzzy sets (e.g.  $A \cap \bar{A} \neq \emptyset$ ).

#### 3. What is a normal fuzzy set?

A fuzzy set is normal if its height is exactly 1.0 (i.e. at least one element has a membership value of 1).

#### 4. Define a subnormal fuzzy set.

A fuzzy set whose maximum membership value (height) is strictly less than 1.0.

**Frequently Asked / Viva Questions (contd.)****5. Why is Centroid defuzzification the most commonly used method?**

Because it integrates the entire area of the output fuzzy shape, yielding smooth, continuous output transitions relative to changes in inputs.

**6. Explain the Max-Min composition rule.**

It is a method of relating elements of set  $X$  to set  $Z$  through intermediate set  $Y$ . For each  $(x, z)$  pair, it finds the strength of paths through all  $y \in Y$  (using Min) and selects the strongest path (using Max).

**7. What is the difference between a tolerance relation and an equivalence relation?**

An equivalence relation must be reflexive, symmetric, and transitive. A tolerance relation is only reflexive and symmetric (not transitive).

**8. Define the support of a fuzzy set.**

The crisp set of all elements in the universe of discourse whose membership values are strictly greater than 0 ( $\mu_A(x) > 0$ ).

**9. What are the standard forms of membership functions?**

Triangular, Trapezoidal, Gaussian, Sigmoidal, L-shaped, and S-shaped curves.

**10. What is the purpose of defuzzification in fuzzy controllers?**

Physical actuators and microcontrollers require crisp, exact values (like voltage or valve position) to operate, which requires defuzzification of the fuzzy control decision.

**Crisp Sets vs. Fuzzy Sets**

| Attribute         | Classical (Crisp) Sets              | Fuzzy Sets                                     |
|-------------------|-------------------------------------|------------------------------------------------|
| Membership Values | Binary: $\{0, 1\}$                  | Continuous: $[0, 1]$                           |
| Boundaries        | Sharp, distinct boundaries          | Gradual, vague boundaries                      |
| Operations        | Boolean logic (AND, OR, NOT)        | Fuzzy logic (Min, Max, $1 - \mu$ )             |
| Contradiction Law | Holds: $A \cap \bar{A} = \emptyset$ | Does not hold: $A \cap \bar{A} \neq \emptyset$ |

# Module 5

## Applications of Artificial Neural Networks

---

### 1. Applications of Artificial Neural Networks

---

Neural networks excel at pattern recognition, function approximation, and adaptive modeling due to their parallel distributed architecture and learning capabilities.

1. **Pattern and Character Recognition:** ANNs (particularly CNNs and MLPs) are widely used for Optical Character Recognition (OCR), signature verification, and face recognition by learning translation-invariant spatial features.
2. **Image Compression:** Feedforward networks with a narrow hidden layer (bottleneck autoencoders) perform dimensionality reduction. The input image is reconstructed at the output layer, while the compressed representation is stored at the hidden layer.
3. **Communication Systems (Channel Equalization):** Equalizers modeled using neural networks (like radial basis function networks) reduce Inter-Symbol Interference (ISI) in dispersive channels by acting as non-linear classifiers.
4. **Control Systems (System Identification):** ANNs model the forward and inverse dynamics of complex, non-linear chemical and mechanical processes, facilitating adaptive and predictive control schemes.

### 2. Applications of Fuzzy Logic

---

Fuzzy logic is highly effective in systems involving linguistic variables and human decision-making processes, where exact mathematical models are unavailable.

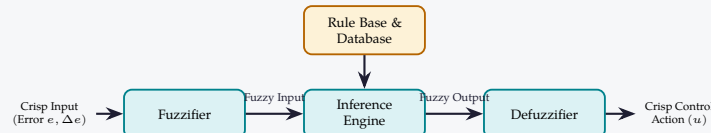
- **Fuzzy Pattern Recognition:** Classifies inputs into overlapping classes based on membership values rather than crisp thresholds.
- **Fuzzy Image Compression:** Uses fuzzy rules to adaptively select quantization levels across different image blocks based on local edge/texture complexity.

### 3. Fuzzy Logic Controllers (FLC)

---

A Fuzzy Logic Controller (FLC) implements a control algorithm by evaluating heuristic, rule-based statements rather than mathematical differential equations.

### Fuzzy Logic Controller Block Diagram



### 3.1 Standard FLC Architecture Components

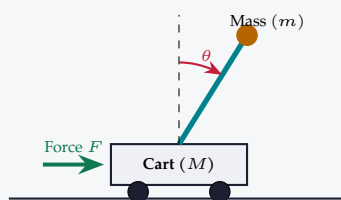
1. **Fuzzifier (Fuzzification Interface):** Measures physical inputs and converts them into fuzzy values (linguistic terms like \*Positive Big\*, \*Zero\*, \*Negative Small\*) using predefined input membership functions.
2. **Knowledge Base (Rule & Database):** Contains the database (definitions of membership functions) and the Rule Base (a collection of linguistic control statements in **IF-THEN** format).
3. **Inference Engine (Decision Making Logic):** Evaluates the active rules to determine the overall fuzzy control output based on composition rules (usually Max-Min composition).
4. **Defuzzifier (Defuzzification Interface):** Converts the combined fuzzy output into a single crisp control command to drive physical actuators.

## 4. Control System Case Studies

### 4.1 Inverted Pendulum Controller

The goal is to maintain a rotating pole in an upright position by moving a motorized cart horizontally.

#### Inverted Pendulum Physical Model



- **Inputs:** Angle error  $\theta$  (deviation from vertical) and angular velocity  $e_{\omega} = d\theta/dt$ .
- **Output:** Motor force  $F$  applied to the cart.
- **Rule Base example:**
  - IF angle ( $\theta$ ) is Positive Small AND angular velocity ( $d\theta/dt$ ) is Zero, THEN force ( $F$ ) is Positive Small.

### 4.2 Fuzzy Washing Machine Controller

An FLC optimizes the wash cycle time, water temperature, and spin speed based on:

- **Inputs:** Dirtiness level of clothes (using optical sensors that measure water transparency) and weight of load.
- **Output:** Cycle duration (wash time).
- **Rule Base example:**
  - IF dirtiness is High AND weight is Medium, THEN wash time is Long.

## 5. Worked Examples

### Fuzzy Inference System Evaluation

**Problem:** Consider a simple Fuzzy Logic Controller with two input variables  $x$  (error) and  $y$  (change of error), and one output  $z$  (control force). The fuzzy rule base contains the following two rules:

- **Rule 1:** IF  $x$  is  $A_1$  AND  $y$  is  $B_1$ , THEN  $z$  is  $C_1$
- **Rule 2:** IF  $x$  is  $A_2$  AND  $y$  is  $B_2$ , THEN  $z$  is  $C_2$

Given crisp input values  $x_0 = 4$  and  $y_0 = 6$ :

- Membership values:  $\mu_{A_1}(x_0) = 0.6$ ,  $\mu_{A_2}(x_0) = 0.3$ ,  $\mu_{B_1}(y_0) = 0.8$ ,  $\mu_{B_2}(y_0) = 0.4$
- Output membership centers (represented by singletons):  $z_{C_1} = 20$ ,  $z_{C_2} = 50$

Evaluate the output control value  $z^*$  using the **weighted average** defuzzification method.

**Solution:**

**1. Compute Firing Strengths ( $\alpha$ -levels) of the rules:** Since the connective is **AND**, we use the Min operator:

- For **Rule 1:**

$$\alpha_1 = \min(\mu_{A_1}(x_0), \mu_{B_1}(y_0)) = \min(0.6, 0.8) = 0.6$$

- For **Rule 2:**

$$\alpha_2 = \min(\mu_{A_2}(x_0), \mu_{B_2}(y_0)) = \min(0.3, 0.4) = 0.3$$

**2. Calculate Defuzzified Output ( $z^*$ ):** Using the weighted average formula:

$$\begin{aligned} z^* &= \frac{\alpha_1 z_{C_1} + \alpha_2 z_{C_2}}{\alpha_1 + \alpha_2} \\ &= \frac{0.6(20) + 0.3(50)}{0.6 + 0.3} \\ &= \frac{12 + 15}{0.9} = \frac{27}{0.9} = 30 \end{aligned}$$

The output control force is  $z^* = 30$ .

## Quick Revision — Module V

Applications of Neural Networks & Fuzzy Logic | PE-EC702C

### Key Formulas at a Glance

**Rule Firing Strength (AND)**  $\alpha_i = \min(\mu_{A_i}(x_0), \mu_{B_i}(y_0))$

**Rule Firing Strength (OR)**  $\alpha_i = \max(\mu_{A_i}(x_0), \mu_{B_i}(y_0))$

**Weighted Average Defuzzification**  $z^* = \frac{\sum \alpha_i z_i}{\sum \alpha_i}$

**Inverted Pendulum Angle Error**  $e(t) = \theta_{ref} - \theta(t)$

**Inverted Pendulum Error Change**  $\Delta e(t) = e(t) - e(t-1)$

### One-Line Definitions

- **Fuzzy Controller:** A control device that uses linguistic rules and fuzzy reasoning to calculate system control inputs.
- **Fuzzifier:** The interface in an FLC that maps physical crisp sensor readings to linguistic fuzzy values.
- **Inference Engine:** The decision-making unit that evaluates active rule statements to determine control signals.
- **Linguistic Variable:** A variable whose values are words or sentences in a language rather than numbers (e.g. "Warm").
- **System Identification:** The process of developing a mathematical or neural network model to describe dynamic process behavior.

### Frequently Asked / Viva Questions

1. **What are the main advantages of a Fuzzy Logic Controller over a PID controller?**  
 FLCs do not require an exact mathematical model of the process, can handle non-linear dynamics easily, and can incorporate human operator heuristics directly using linguistic rules.
2. **State the function of the FLC Rule Base.**  
 The Rule Base contains the control logic written as a set of IF-THEN rules that describe the relationship between input error conditions and output control actions.
3. **How is a neural network used for image compression?**  
 An autoencoder network is trained to replicate inputs at its output layer. A bottleneck hidden layer (fewer nodes) forces the network to learn a compressed representation of the image.

**Frequently Asked / Viva Questions (contd.)****4. What is channel equalization in communications, and how do ANNs help?**

Channel equalization compensates for signal distortion (ISI) caused by transmission channels. ANNs act as adaptive classifiers that identify non-linear decision boundaries to recover transmitted symbols.

**5. Explain the wash time optimization in a fuzzy washing machine.**

Sensors measure the quantity of load and the dirtiness of the water (transparency). The FLC infers the optimum wash cycle time: muddy water with a heavy load triggers a long wash cycle.

**6. What inputs are required to balance an inverted pendulum?**

The deviation angle  $\theta$  from the vertical axis (error) and the rate of change of the angle  $d\theta/dt$  (angular velocity).

**7. What is the difference between Mamdani and Sugeno fuzzy inference systems?**

Mamdani systems use fuzzy sets for output membership functions, requiring defuzzification. Sugeno systems use linear mathematical functions or constants for outputs, simplifying calculation.

**8. Define a Singleton membership function.**

A singleton is a membership function that has a value of 1.0 at a single specific point and 0 everywhere else.

**9. How does a neural network model non-linear control systems?**

By acting as a system identification model, learning the mapping between system input history and corresponding output responses.

**10. What is the role of the database in a fuzzy controller?**

The database holds the exact parameters (shapes, coordinates, limits) of the membership functions defined for the input and output variables.

**Comparison of Control Strategies**

| Parameter           | Classical PID Control                    | Fuzzy Logic Control                      |
|---------------------|------------------------------------------|------------------------------------------|
| Process Model       | Requires exact mathematical model        | No mathematical model required           |
| Control Logic       | Defined by differential equations        | Defined by linguistic IF-THEN rules      |
| Tuning Parameters   | Proportional, Integral, Derivative gains | Membership functions and rule parameters |
| Non-linear Handling | Difficult (often requires linearization) | Inherently handles non-linearities       |